

Sri Lanka Institute of Information Technology

B.Sc. (Hons) Information Technology
Cyber security

Building and Securing a DevSecOps Pipeline



IE3142- DevOps Security
Year 3 Semester 1- 2026

Group ID – 64

1. Table of Contents

2.	Executive Summary	4
3.	System Overview	4
3.1.	Application and technology stack	4
3.2.	Architecture and trust boundaries	5
3.3.	Containerization and execution	7
4.	Threat model and Risk Assessment	7
4.1.	Threat Modelling Methodology	7
4.2.	Assets	7
4.3.	STRIDE Threat Model	8
4.4.	Risk Assessment	8
4.5.	Detailed Threat Analysis	10
4.5.1.	T1 -SQL Injection	10
4.5.2.	T2 - Unauthenticated Service Request Access	11
4.5.3.	T3 - Server-Side Request Forgery	13
4.5.4.	T4 - Broken Object Level Authorization (BOLA/IDOR)	16
4.6.	Threat-to-Control Mapping	18
5.	Secure Coding Evidence	18
5.1.	SQL Injection - Exploit and Fix	18
5.2.	Unauthenticated Service Request - Exploit and Fix	20
5.3.	SSRF - Exploit and Fix	21
5.4.	BOLA/IDOR - Exploit and Fix	23
6.	CI/CD Pipeline and Security Automation	24
6.1.	Pipeline Overview	24
6.2.	Build and Application Validation	25
6.3.	SAST - Semgrep	26
6.4.	Trivy Dependency / Software Composition Scanning	27
6.5.	Secrets Scanning - Gitleaks	28
6.6.	Container Image Scanning - Trivy	29
6.7.	CI/CD Security Summary	30
7.	Secrets Management	31
7.1.	Local Development Secrets	31
7.2.	GitHub Actions Secrets	31
7.3.	Secret Injection	31
8.	Industry Trend and Case Study	32
8.1.	SolarWinds Supply-Chain Compromise	32
8.2.	Lessons Applied to This Project	33

9.	Reflection and Future Improvements	33
10.	Contributions and AI Use	34
11.	Conclusion.....	34
12.	References.....	35

2. Executive Summary

This group project applies DevSecOps practices to OWASP crAPI, an intentionally vulnerable API-driven application. The group selected four application-specific weaknesses for assessment: SQL injection in the coupon endpoint, unauthenticated access to a mechanic service request, server-side request forgery (SSRF) through the mechanic API field, and broken object-level authorization (BOLA) in mechanic reports. The group's baseline notes state that these behaviors were reproduced on an unmodified deployment and describe the data or internal service reached. The repository contains crAPI source, Compose, environment examples, a GitHub Actions workflow, and a Semgrep output reporting nine findings. However, Compose and CI pull published images instead of building the group's modified source. No fixed-version exploit results or before/after SAST comparison were available for this review. The report separates baseline observations from unverified remediation and CI results. The final technical claim depends on reproducing both versions and retesting each exploit against the fixed build.

Source Code Repository:

<https://github.com/Vidumal/DevSecOps-Assignment->

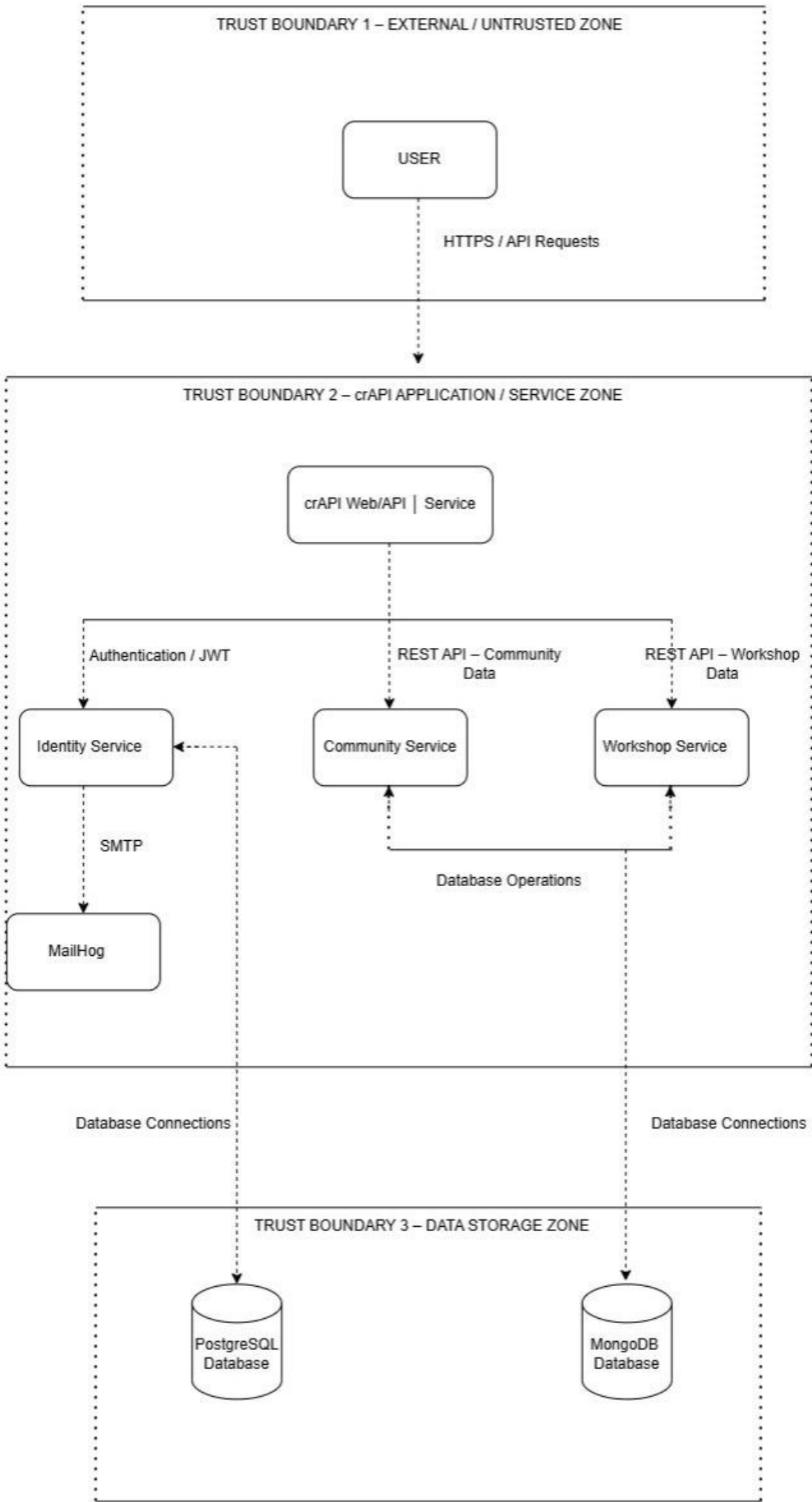
3. System Overview

3.1. Application and technology stack

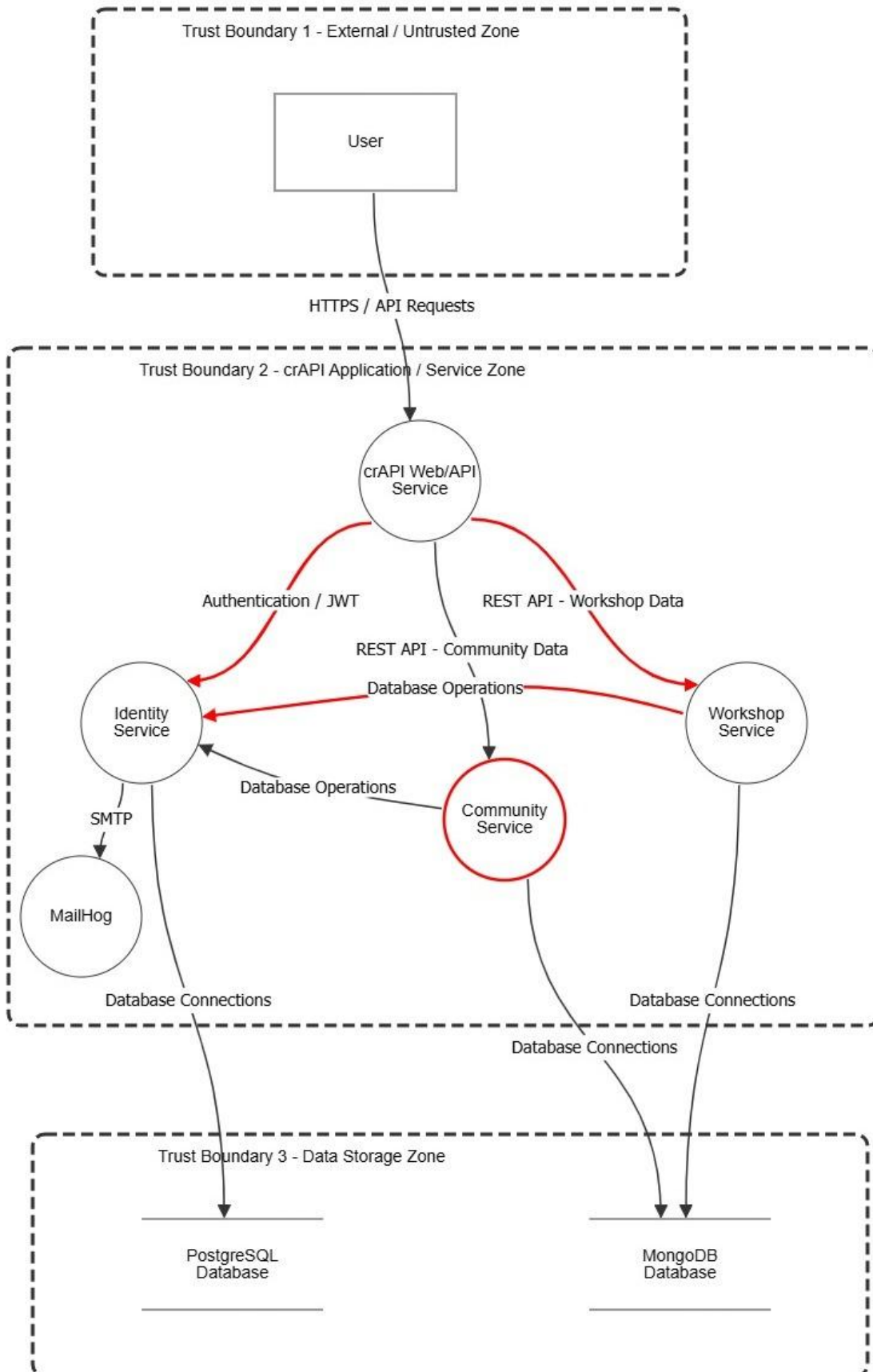
OWASP crAPI (Completely Ridiculous API) is an intentionally vulnerable training application that simulates a vehicle-owner platform implemented as communicating services. OWASP describes it as a microservice-based application focused on API security risks [3]. The selected project is appropriate to the assignment because it provides multiple interacting services, data stores, and internal endpoints, allowing the group to study authorization, input handling, and trust between services.

The project uses Docker Compose, GitHub Actions, Semgrep, Trivy, and Gitleaks. Compose lists a web frontend, gateway, identity, workshop, community, chatbot, PostgreSQL, MongoDB, ChromaDB, and MailHog. Databases and internal tools should not be public entry points

3.2. Architecture and trust boundaries



3.3. STRIDE diagram



3.4. Containerization and execution

The root Compose file uses published images, while crAPI/ contains source. This starts upstream crAPI but does not prove changed code is running. Connect the affected service Dockerfiles/build contexts to Compose and label baseline and fixed images separately. Document prerequisites, startup/shutdown, URLs, and the version selection in README; verify a clean checkout.

4. Threat model and Risk Assessment

4.1. Threat Modelling Methodology

The crAPI application was threat-modelled using the **STRIDE methodology**. The objective was to identify threats affecting authentication, authorization, application data, backend services and communication between microservices.

The assessment focused on four vulnerabilities identified during testing of the unmodified crAPI deployment:

- SQL Injection
- Unauthenticated Service Request Access
- Server-Side Request Forgery
- Broken Object Level Authorization

The threats were selected because they are directly applicable to the application's API architecture and can result in unauthorized data access, unauthorized functionality, manipulation of application data, or access to internal services.

4.2. Assets

The main assets identified in the crAPI environment are:

Asset	Description
User accounts	Authentication credentials and user identity information
Vehicle information	VIN, vehicle details, ownership information and vehicle-related data
Service requests	Mechanic requests, problem descriptions and customer information
Database records	PostgreSQL/MongoDB application data
Internal services	Workshop, identity and other Docker services
Internal network	Docker network containing backend services
API endpoints	Interfaces through which users interact with application functionality
Application secrets	Database credentials, API secrets and service credentials

4.3. STRIDE Threat Model

ID	STRIDE Category	Threat	Related Vulnerability	Affected Component
T1	Tampering / Information Disclosure	Attacker manipulates database query input to retrieve unintended database information	SQL Injection	Workshop API / Database
T2	Spoofing / Elevation of Privilege	Unauthenticated user accesses service-request information	Unauthenticated Service Request Access	Workshop API
T3	Server-Side Request Forgery / Information Disclosure	Attacker causes backend server to make requests to internal services	SSRF	Workshop API / Internal network
T4	Elevation of Privilege / Information Disclosure	Authenticated user accesses another user's service-request object	BOLA/IDOR	Mechanic report API

The SQL injection evidence shows that the `apply_coupon` endpoint accepted SQL-related input and returned database information, demonstrating the potential for database-level impact.

The unauthenticated service-request test demonstrated that `GET /workshop/api/mechanic/service_request/1` returned 200 OK without authentication and exposed mechanic and vehicle-owner information.

The SSRF test demonstrated that the attacker-controlled `mechanic_api` field could be used to reach the internal MailHog service, with the internal service response returned through the API.

The BOLA test demonstrated that an authenticated user could access report ID 1 belonging to another user, with vehicle VIN, email, mobile number and problem details exposed.

4.4. Risk Assessment

Likelihood	
Score	Description
1	Rare
2	Unlikely
3	Possible
4	Likely
5	Almost Certain

Impact	
Score	Description
1	Negligible
2	Minor
3	Moderate
4	Major
5	Severe

Risk Score = Likelihood × Impact

Threat	Likelihood	Impact	Risk Score	Risk Level	Reason
T1 – SQL Injection	4	5	20	Critical	Database query manipulation can expose or modify application data
T2 – Unauthenticated Service Request Access	4	4	16	High	Service-request and vehicle-owner information can be accessed without authentication
T3 – SSRF	4	5	20	Critical	Backend can be induced to access internal services
T4 – BOLA/IDOR	4	4	16	High	Authenticated users can access another user's service-request information

4.5. Detailed Threat Analysis

4.5.1. T1 -SQL Injection

Vulnerability

The affected endpoint is:

```
POST /workshop/api/shop/apply_coupon
```

The evidence document records the SQL-injection payload and states that the response exposed the value 9876540001. The documented root cause is direct concatenation of user input into SQL.

Threat

An attacker can provide crafted input that changes the intended SQL statement.

Impact

Potential consequences include:

- Unauthorized database information disclosure
- Modification of database records
- Authentication-related data exposure
- Potential broader database compromise depending on database privileges

Recommended control

Use parameterized queries/prepared statements rather than constructing SQL statements through string concatenation.

Conceptually:

Unsafe:

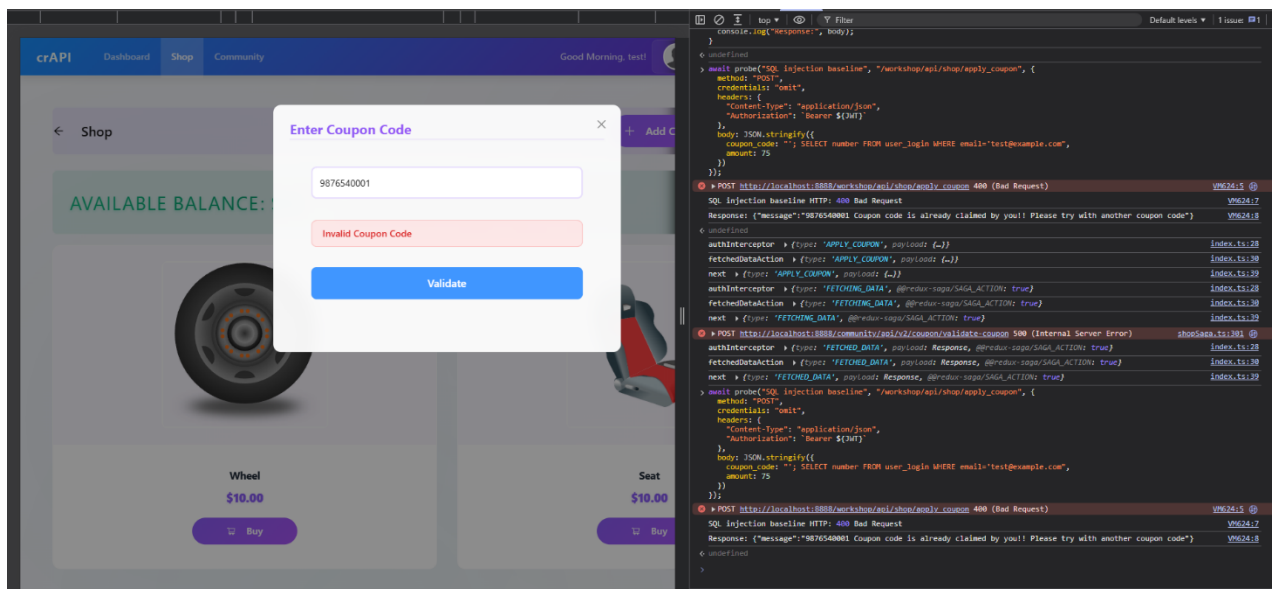
```
"SELECT ... WHERE coupon = '" + userInput + "'"
```

Secure:

```
"SELECT ... WHERE coupon = ?"
```

Exploit code we used:

```
await probe("SQL injection baseline", "/workshop/api/shop/apply_coupon", {
  method: "POST",
  credentials: "omit",
  headers: {
    "Content-Type": "application/json",
    "Authorization": `Bearer ${JWT}`
  },
  body: JSON.stringify({
    coupon_code: ""; SELECT number FROM user_login WHERE email='test@example.com',
    amount: 75
  })
});
```



4.5.2. T2 - Unauthenticated Service Request Access

Vulnerability

Affected endpoint:

```
GET /workshop/api/mechanic/service_request/1
```

The evidence identifies no authentication and records a 200 OK response containing mechanic and vehicle-owner information.

Threat

An unauthenticated attacker can directly request a service-request object without first proving their identity.

Impact

Potential exposure includes:

- Vehicle information
- Customer information
- Mechanic information
- Service-request information

Recommended control

The endpoint should require authentication before processing the request.

A suitable flow is:

Request



Authentication middleware



Valid token?



No Yes

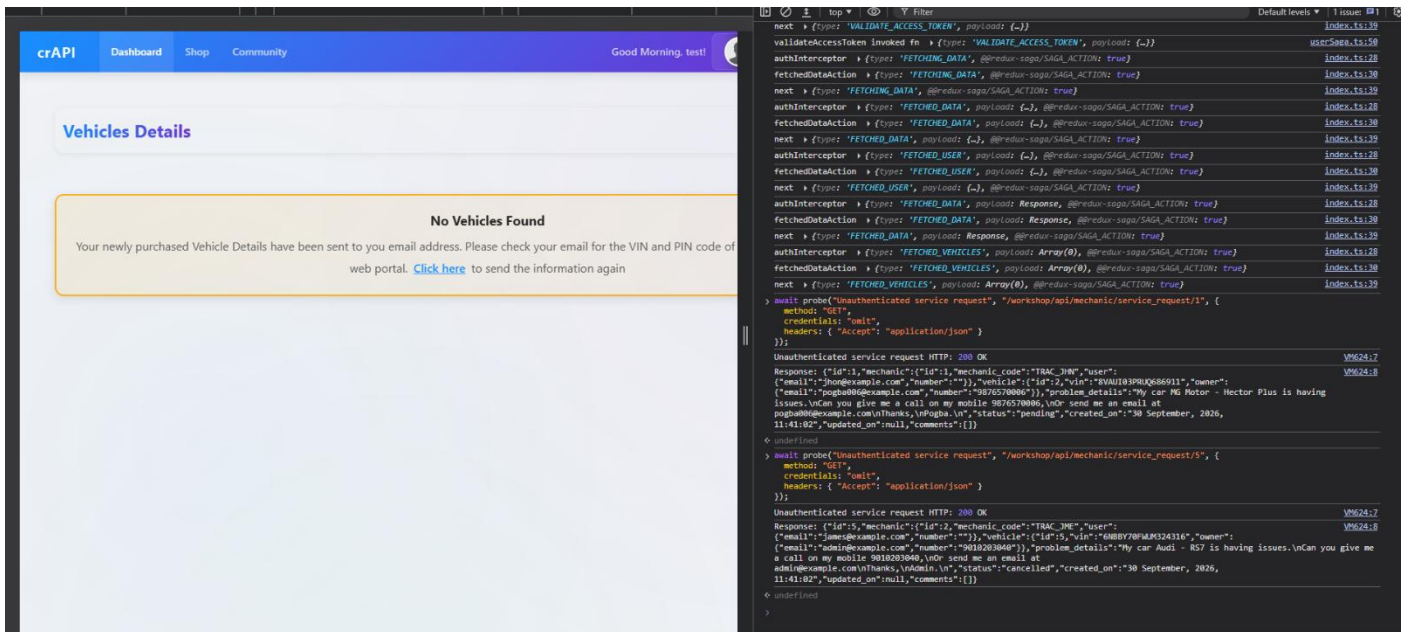


401 Authorization check

The endpoint should reject unauthenticated requests with an appropriate 401 Unauthorized response.

Exploit code we used:

```
await probe("Unauthenticated service request", "/workshop/api/mechanic/service_request/1", {
  method: "GET",
  credentials: "omit",
  headers: { "Accept": "application/json" }
});
```



4.5.3. T3 - Server-Side Request Forgery Vulnerability

Affected endpoint:

POST /workshop/api/merchant/contact_mechanic

The attacker-controlled field is:

mechanic_api

The evidence demonstrates that supplying:

http://mailhog:8025

caused the backend to reach the internal MailHog service and return its HTML response.

Threat

The attacker does not directly access the internal service. Instead:

Attacker



crAPI API



Backend server



Internal service

The backend becomes the attacker's HTTP client.

Impact

Depending on network configuration and available internal services, SSRF can potentially allow:

- Internal service discovery
- Access to internal administrative interfaces
- Access to cloud metadata services in cloud deployments
- Internal network reconnaissance
- Retrieval of sensitive internal responses

Recommended control

Do not allow arbitrary URLs supplied by users.

Use:

- URL allowlisting
- Strict hostname validation
- IP address validation
- Blocking private/internal address ranges
- Blocking loopback addresses
- Blocking link-local addresses
- Restricting outbound network access

For this project, the strongest implementation is to avoid accepting arbitrary `mechanic_api` URLs and instead map approved mechanic services to predefined server-side destinations.

Exploit code we used:

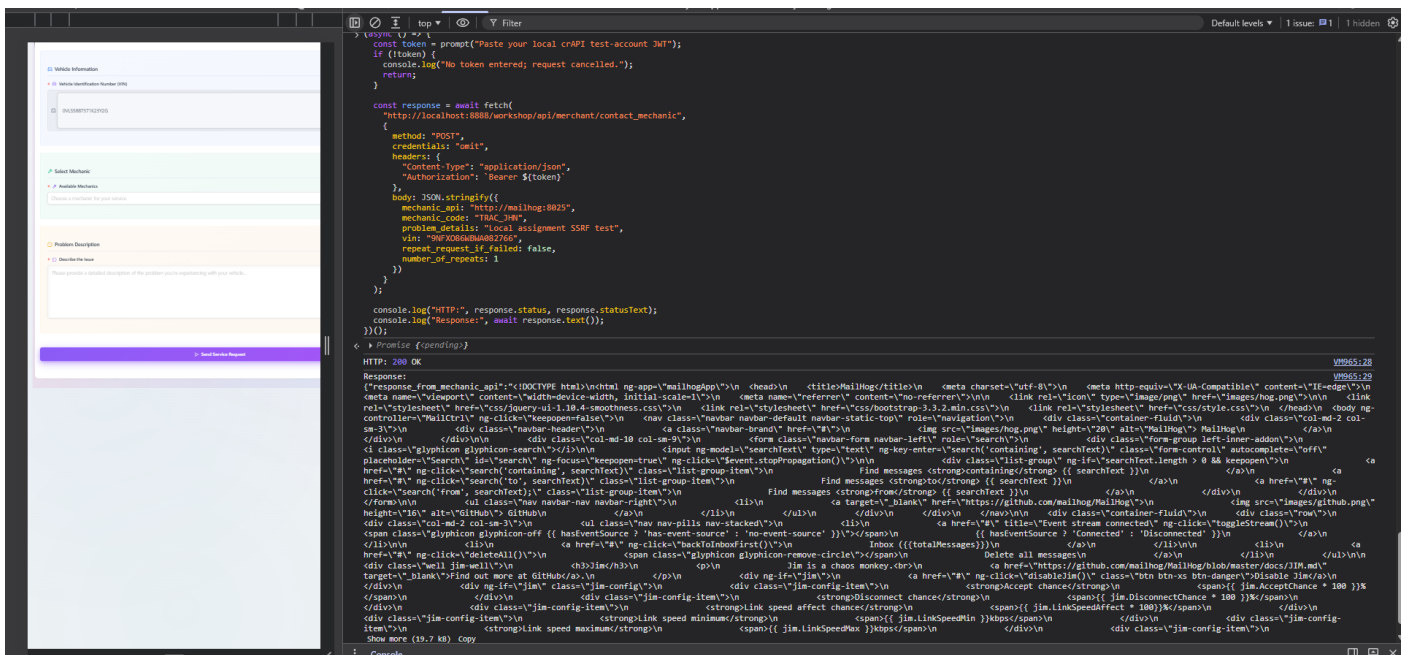
```

(async () => {
  const token = prompt("Paste your local crAPI test-account JWT");
  if (!token) {
    console.log("No token entered; request cancelled.");
    return;
  }

  const response = await fetch(
    "http://localhost:8888/workshop/api/merchant/contact_mechanic",
    {
      method: "POST",
      credentials: "omit",
      headers: {
        "Content-Type": "application/json",
        "Authorization": `Bearer ${token}`
      },
      body: JSON.stringify({
        mechanic_api: "http://mailhog:8025",
        mechanic_code: "TRAC_JHN",
        problem_details: "Local assignment SSRF test",
        vin: "9NFX086WBWA082766",
        repeat_request_if_failed: false,
        number_of_repeats: 1
      })
    }
  );

  console.log("HTTP:", response.status, response.statusText);
  console.log("Response:", await response.text());
})();

```



4.5.4. T4 - Broken Object Level Authorization (BOLA/IDOR)

Vulnerability

Affected endpoint:

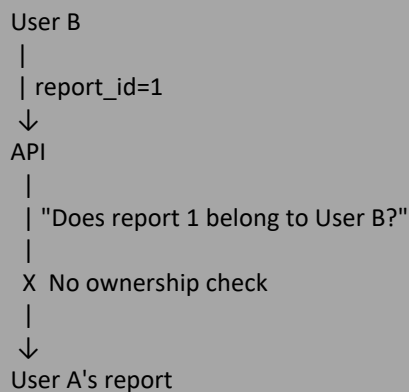
```
GET /workshop/api/mechanic/mechanic_report?report_id=1
```

The evidence shows that authenticated user `test@example.com` accessed report ID 1, while the actual object owner was `adam007@example.com`. The response exposed vehicle VIN, email, mobile number and problem details.

Threat

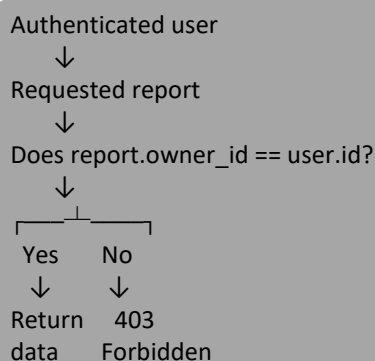
The server uses a client-controlled object identifier without verifying that the authenticated user owns or is authorized to access the requested object.

Conceptually:



Recommended control

Perform a server-side ownership/authorization check:



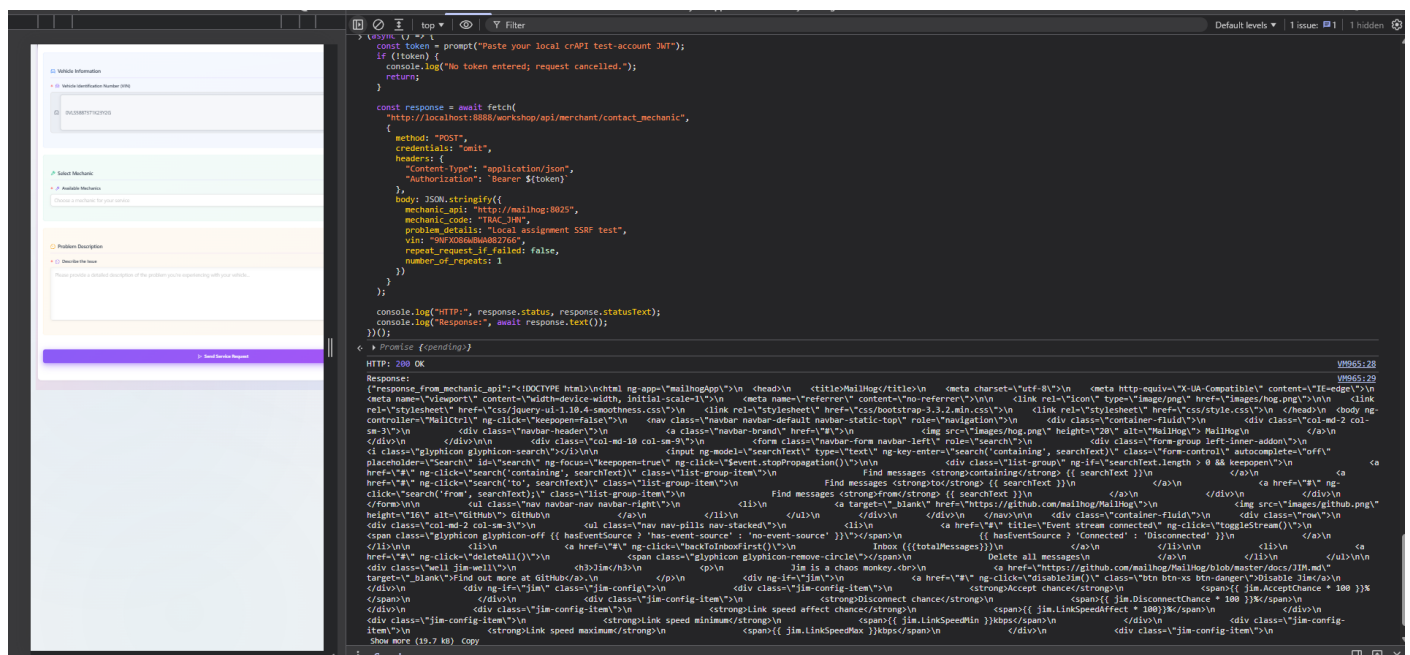
The authorization decision must occur on the server and must never rely solely on the object ID supplied by the client.

Exploit code we used:

```
(async () => {
  const token = prompt("Paste your local crAPI test-account JWT");
  if (!token) {
    console.log("No token entered; request cancelled.");
    return;
  }

  const response = await fetch(
    "http://localhost:8888/workshop/api/mechanic/mechanic_report?report_id=1",
    {
      method: "GET",
      credentials: "omit",
      headers: {
        "Accept": "application/json",
        "Authorization": `Bearer ${token}`
      }
    }
  );

  console.log("HTTP:", response.status, response.statusText);
  console.log("Response:", await response.text());
});
```



4.6. Threat-to-Control Mapping

Threat	Vulnerability	Primary Control	Expected Result
T1	SQL Injection	Parameterized queries / prepared statements	User input cannot alter SQL structure
T2	Unauthenticated Service Request	Authentication middleware	Unauthenticated requests receive 401
T3	SSRF	URL allowlisting + private IP blocking + outbound restrictions	Arbitrary internal destinations cannot be reached
T4	BOLA/IDOR	Object-level authorization / ownership validation	Users cannot access another user's objects

5. Secure Coding Evidence

This section should document the **exploit → code fix → same exploit re-tested** workflow.

5.1. SQL Injection - Exploit and Fix

Test	Before Fix	After Fix
Payload	' ; SELECT number FROM user_login WHERE email='test@example.com	Same exact payload
Status	HTTP 400	HTTP 400
Result	Exposed 9876540001	Coupon not found
SQL injection executed	Yes	No evidence of execution
Sensitive data exposed	Yes	No

Security Fix:

The query was changed to use parameterized database access. User-controlled input is passed as a parameter rather than concatenated into the SQL statement.

Retest:

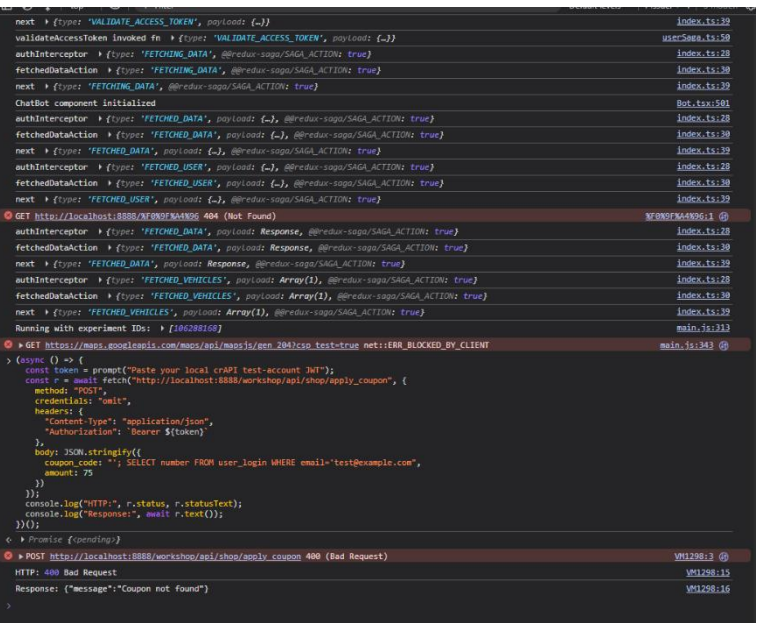
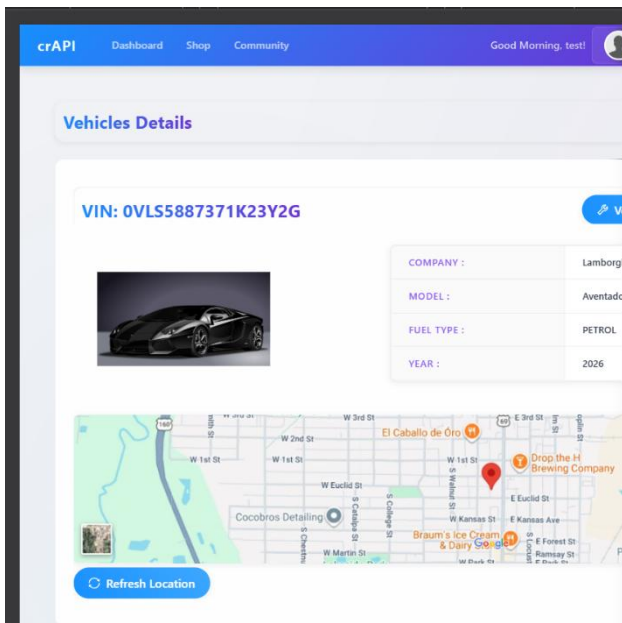
The exact baseline payload was submitted against the rebuilt application. The request was rejected as an invalid coupon and no database value was returned. This demonstrated that the original SQL injection condition was no longer reproducible.

```

PS C:\Users\Pramuditha\Desktop\DevSecOps-Assignment--main\DevSecOps-Assignment--mai
h\test run\crAPI-main\deploy\docker> $body = @{
    coupon_code = ""; SELECT number FROM user_login WHERE email='test@example.co
m"
>
    amount = 75
> } | ConvertTo-Json
PS C:\Users\Pramuditha\Desktop\DevSecOps-Assignment--main\DevSecOps-Assignment--mai
h\test run\crAPI-main\deploy\docker>
PS C:\Users\Pramuditha\Desktop\DevSecOps-Assignment--main\DevSecOps-Assignment--mai
h\test run\crAPI-main\deploy\docker> Invoke-RestMethod `
> -Method Post `
> -Uri "http://localhost:8888/workshop/api/shop/apply_coupon" `
> -Headers $headers `
> -ContentType "application/json" `
> -Body $body
Invoke-RestMethod : The remote server returned an error: (400) Bad Request.
At line:1 char:1
+ Invoke-RestMethod `
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest
:HttpWebRequest) [Invoke-RestMethod], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.Powe
rShell.Commands.InvokeRestMethodCommand

PS C:\Users\Pramuditha\Desktop\DevSecOps-Assignment--main\DevSecOps-Assignment--mai
h\test run\crAPI-main\deploy\docker> try {
>     Invoke-RestMethod `
>     -Method Post `
>     -Uri "http://localhost:8888/workshop/api/shop/apply_coupon" `
>     -Headers $headers `
>     -ContentType "application/json" `
>     -Body $body
> }
> catch {
>     $reader = New-Object System.IO.StreamReader($_.Exception.Response.GetResponse
Stream())
>     $reader.ReadToEnd()
> }
{"message":"Coupon not found"}
PS C:\Users\Pramuditha\Desktop\DevSecOps-Assignment--main\DevSecOps-Assignment--mai
h\test run\crAPI-main\deploy\docker> |

```



5.2. Unauthenticated Service Request - Exploit and Fix

Test	Before Fix	After Fix
Endpoint	GET /workshop/api/mechanic/mechanic_report?report_id=1	Same
Authenticated user	test@example.com	Same
report_id	1	1
Result	200 OK + another user's report	Report does not exist
Other user's data exposed	Yes	No

Root Cause:

The endpoint did not enforce authentication before processing the service-request lookup. As a result, possession of the endpoint URL was sufficient to retrieve the object.

Security Fix:

Authentication enforcement was added so that the endpoint requires a valid authenticated user before processing the request.

Retest:

The same unauthenticated request was sent against the rebuilt application. The server rejected the request with HTTP 401 Unauthorized and did not return the service-request data.

The screenshot displays a web application interface on the left and a browser console log on the right. The web application, titled 'crAPI', shows 'Vehicles Details' for a specific VIN: 0VLS5887371K23Y2G. It includes a car image, a map, and a table with vehicle specifications: COMPANY (Lamborghini), MODEL (Aventador), FUEL TYPE (PETROL), and YEAR (2026). The browser console on the right shows a series of Redux state updates and a failed HTTP request. The failed request is a GET to 'http://localhost:8888/workshop/api/mechanic/service_request/1' which returned a 401 Unauthorized status. The response message is 'JWT Token required!'.

5.3. SSRF - Exploit and Fix

Item	Result
Vulnerability	SSRF
Endpoint	POST /workshop/api/merchant/contact_mechanic
Original payload	http://mailhog:8025
Original result	MailHog internal HTML returned
Fix	Validate/deny unsafe destination URLs
Same payload after fix	Rejected
HTTP result	400 Bad Request
Status	Fixed & verified

Root Cause:

The endpoint accepted a user-controlled URL and used it as a backend destination without sufficiently restricting the destination. Because the backend container can communicate with internal Docker services, an attacker could use the API as a proxy to reach an internal service.

Security Fix:

The implementation was changed to reject arbitrary destinations and enforce destination validation. Requests to internal, loopback, link-local and private network destinations are rejected. Where possible, approved mechanic destinations are represented by predefined server-side values rather than accepting arbitrary URLs from the client.

```

PS C:\Users\Pramuditha\Desktop\DevSecOps-Assignment--main\DevSecOps-Assignment--main\test r
un\crAPI-main\deploy\docker> docker compose up -d --force-recreate crapi-workshop
[+] up 6/6
✔ Container crapi-community Healthy 9.6s
✔ Container postgresdb Healthy 9.6s
✔ Container crapi-identity Healthy 9.6s
✔ Container mongodb Healthy 9.6s
✔ Container mailhog Healthy 8.6s
✔ Container crapi-workshop Started 10.0s
PS C:\Users\Pramuditha\Desktop\DevSecOps-Assignment--main\DevSecOps-Assignment--main\test r
un\crAPI-main\deploy\docker> docker inspect crapi-workshop --format '{{.State.Status}} | Ex
itCode={{.State.ExitCode}}'
running | ExitCode=0
PS C:\Users\Pramuditha\Desktop\DevSecOps-Assignment--main\DevSecOps-Assignment--main\test r
un\crAPI-main\deploy\docker> $ssrfBody = @{
>> mechanic_api = "http://mailhog:8025"
>> repeat_request_if_failed = $false
>> mechanic_code = "anything"
>> vin = "anything"
>> problem_details = "SSRF test"
>> } | ConvertTo-Json
PS C:\Users\Pramuditha\Desktop\DevSecOps-Assignment--main\DevSecOps-Assignment--main\test r
un\crAPI-main\deploy\docker> try {
>> Invoke-RestMethod `
>> -Method Post `
>> -Uri "http://localhost:8888/workshop/api/merchant/contact_mechanic" `
>> -Headers $headers `
>> -ContentType "application/json" `
>> -Body $ssrfBody
>> }
>> catch {
>> $reader = New-Object System.IO.StreamReader($_.Exception.Response.GetResponseStream(
))
>> $reader.ReadToEnd()
>> }
{"message":"Invalid or unsafe mechanic API URL."}
PS C:\Users\Pramuditha\Desktop\DevSecOps-Assignment--main\DevSecOps-Assignment--main\test r
un\crAPI-main\deploy\docker> |

```

The screenshot displays the crAPI web application in a browser window. The page title is "Vehicles Details" and it shows information for a vehicle with VIN: 0VLS5887371K23Y2G. The details include the company (Lamborghini), model (Aventador), fuel type (PETROL), and year (2026). Below the details is a map showing the location. A "Refresh Location" button is visible at the bottom left of the map area.

The developer console on the right shows network requests. A GET request to `http://localhost:8888/workshop/api/merchant/contact_mechanic` is highlighted, showing a 400 Bad Request response. The response body is `{"message":"Invalid mechanic API endpoint."}`. The console also shows other requests, including a GET request to `http://localhost:8888/workshop/api/merchant/contact_mechanic` and a POST request to `http://localhost:8888/workshop/api/merchant/contact_mechanic`.

5.4. BOLA/IDOR - Exploit and Fix

Item	Details
Vulnerability	Broken Object Level Authorization (BOLA / IDOR)
Affected Endpoint	GET /workshop/api/mechanic/mechanic_report?report_id=1
Before Exploitation	An authenticated user could request another user's service report by changing/providing the report_id. The original code retrieved the service request using only the supplied ID, without verifying that it belonged to the authenticated mechanic.
Exploit	Logged in as test@example.com and requested report_id=1, which belonged to another user (adam007@example.com).
Evidence Before Fix	HTTP 200 OK returned the other user's report, including VIN, email, phone number, vehicle information and problem details.
Impact	Unauthorized users could access service-request information belonging to other users, causing sensitive personal and vehicle information disclosure.
Security Fix	The query was changed to verify both the requested ID and the authenticated mechanic: ServiceRequest.objects.filter(id=report_id, mechanic__user=user).first()
After Exploitation	The exact same request returned 404 Not Found with {"message":"The Report does not exist for given report_id."} instead of the other user's report.

The screenshot displays a web application interface on the left and a browser's developer tools on the right.

Web Application Interface:

- Header: crAPI, Dashboard, Shop, Community, Good Morning, test!
- Section: Vehicles Details
- VIN: 0VLS5887371K23Y2G
- Image: A black sports car.
- Details:
 - COMPANY: Lamborghini
 - MODEL: Aventador
 - FUEL TYPE: PETROL
 - YEAR: 2026
- Map: A map showing the location of the vehicle with a red pin. Landmarks include "El Caballo de Oro", "Drop the H Brewing Company", "Cocobros Detailing", and "Braum's Ice Cream & Dairy Store".
- Button: Refresh Location

Browser Console:

- REST Client Request:


```
GET http://localhost:8888/workshop/api/merchant/contact_mechanic 400 (Not Found)
```
- REST Client Response:


```
HTTP: 400 Bad Request
Response: {"message": "Invalid mechanic API endpoint."}
```
- Console Log:


```
> (async () => {
  const token = prompt("Paste your local crAPI test-account JWT");
  if (token) {
    console.log("No token entered; request cancelled.");
    return;
  }

  const r = await fetch(
    "http://localhost:8888/workshop/api/merchant/contact_mechanic",
    {
      method: "POST",
      credentials: "omit",
      headers: {
        "Content-Type": "application/json",
        "Authorization": "Bearer ${token}"
      },
      body: JSON.stringify({
        mechanic_api: "http://mailhog:8025",
        mechanic_code: "TMC-001",
        problem_details: "Local assignment SSRF verification",
        vin: "0VLS5887371K23Y2G",
        repeat_request_if_failed: false,
        number_of_repeats: 1
      })
    }
  );

  console.log("HTTP:", r.status, r.statusText);
  console.log("Response:", await r.text());
})();
```

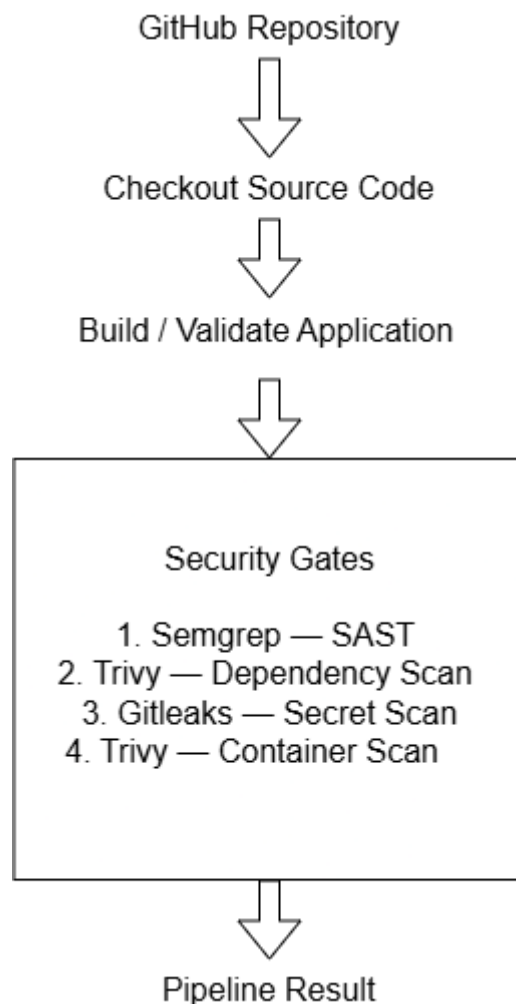
6. CI/CD Pipeline and Security Automation

6.1. Pipeline Overview

A GitHub Actions-based CI/CD security pipeline was implemented to automate application validation and security checks for the crAPI application. The pipeline is triggered automatically when changes are pushed to the main or develop branches and when pull requests are opened against the main branch. A workflow_dispatch trigger is also provided to allow the pipeline to be executed manually when required.

The pipeline performs source-code checkout, application validation, and multiple automated security checks. Four security controls were integrated into the workflow: Semgrep for Static Application Security Testing (SAST), Trivy for dependency vulnerability scanning, Gitleaks for secret detection, and Trivy for Docker container image scanning.

The overall pipeline can be represented as follows:



The workflow is configured to run on:

- Push to main
- Push to develop
- Pull request to main
- Manual execution using workflow_dispatch

The use of automated security gates allows security checks to be performed consistently whenever source-code changes are introduced rather than relying only on manual security testing.

6.2. Build and Application Validation

The pipeline begins by checking out the source code from the GitHub repository using GitHub Actions. Docker Compose is then validated to ensure that the application's Compose configuration is syntactically valid before security scanning is performed.

The repository contains the crAPI service source code and Dockerfiles. In particular, the workshop service containing the selected application vulnerabilities has its own Dockerfile located at:

```
crAPI/services/workshop/Dockerfile
```

The workshop Dockerfile uses Python 3.11 and packages the workshop service into a Docker image.

During the initial implementation, the Compose configuration referenced the published image:

```
crapi/crapi-workshop:latest
```

and the pipeline used:

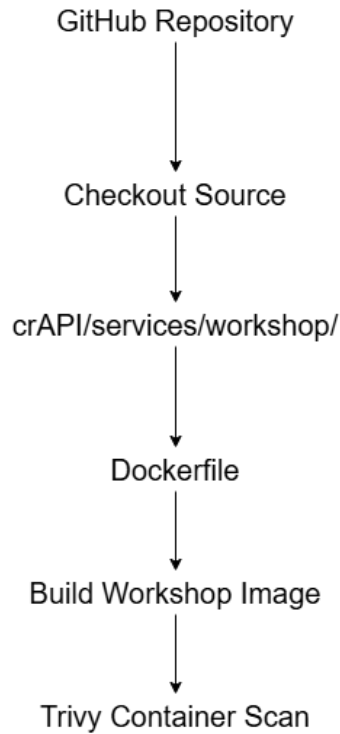
```
docker compose pull
```

This approach downloads the upstream crAPI image and therefore does not guarantee that the Docker image being scanned contains modifications made to the application's source code.

For the final DevSecOps implementation, the affected service is required to be built from the checked-out repository source before the resulting image is passed to the container security scanner. This ensures that the security pipeline evaluates the same application version that has been modified and tested by the group

This approach downloads the upstream crAPI image and therefore does not guarantee that the Docker image being scanned contains modifications made to the application's source code.

For the final DevSecOps implementation, the affected service is required to be built from the checked-out repository source before the resulting image is passed to the container security scanner. This ensures that the security pipeline evaluates the same application version that has been modified and tested by the group.



6.3. SAST - Semgrep

Tool and Purpose

Semgrep was integrated into the pipeline to perform Static Application Security Testing (SAST). SAST examines the application source code for insecure coding patterns without requiring the application to be executed.

The workflow uses the Semgrep security-audit ruleset:

```
- name: Run Semgrep
  uses: semgrep/semgrep-action@v1
  with:
    config: p/security-audit
```

The purpose of this gate is to identify security weaknesses directly in the source code and prevent code containing blocking security findings from progressing through the pipeline.

Baseline Result

During the initial pipeline execution, Semgrep scanned 1,139 tracked files using 225 code rules and identified 9 blocking findings. The findings included:

- Flask application configured to listen on an insecure host configuration
- Use of math/rand instead of a cryptographically secure random generator
- Spring Security CSRF protection disabled
- Insecure hostname verification
- Disabled TLS certificate verification using verify=False

- Django @csrf_exempt usage
- Other disabled certificate-validation controls

The Semgrep workflow completed its scan successfully, but because blocking findings were detected, it returned exit code 1. Therefore, GitHub Actions marked the security job as failed.

The important distinction is that the scanner itself did not crash. The non-zero exit code represents the intended security-gate behaviour.

6.4. Trivy Dependency / Software Composition Scanning

Trivy filesystem scanning was integrated to identify known vulnerabilities in third-party application dependencies.

The relevant configuration is:

```
scan-type: fs
scan-ref: .
scanners: vuln
severity: HIGH,CRITICAL
ignore-unfixed: true
exit-code: '1'
```

The filesystem scan examines dependency manifests contained in the repository, including Python requirements files, Go module files and the web application's npm package-lock file.

The severity threshold was configured to identify **HIGH** and **CRITICAL** vulnerabilities. The ignore-unfixed option prevents vulnerabilities without an available fix from being treated as actionable by this particular gate.

Baseline Result

The initial scan identified **127 vulnerabilities** across the dependency files scanned by Trivy.

The reported vulnerabilities were distributed across:

Dependency source	Vulnerabilities
chatbot/requirements.txt	13
community/go.mod	19
gateway-service/go.mod	0
web/package-lock.json	78
workshop/requirements.txt	17
Total	127

6.5. Secrets Scanning - Gitleaks

Gitleaks was integrated into the pipeline to detect accidentally committed credentials, API keys, passwords and other sensitive information.

The workflow uses:

```
- name: Checkout repository
  uses: actions/checkout@v6
  with:
    fetch-depth: 0
```

The complete Git history is made available to the scanner so that committed secrets can be detected rather than scanning only the latest working tree.

The Gitleaks action is then executed using the repository's GitHub Actions token:

```
- name: Run Gitleaks
  uses: gitleaks/gitleaks-action@v3
  env:
    GITHUB_TOKEN: ${
      secrets.GITHUB_TOKEN }
```

The complete Git history is made available to the scanner so that committed secrets can be detected rather than scanning only the latest working tree.

The Gitleaks action is then executed using the repository's GitHub Actions token:

```
- name: Run Gitleaks
  uses: gitleaks/gitleaks-action@v3
  env:
    GITHUB_TOKEN: ${
      secrets.GITHUB_TOKEN }
```

Baseline Result

The initial Gitleaks scan completed successfully. The scan reported:

```
1 commits scanned
no leaks found
No leaks detected
```

6.6. Container Image Scanning - Trivy

Trivy was also integrated as a container image security scanner. Unlike the filesystem scan, this gate examines the Docker image itself and can identify vulnerabilities in both operating-system packages and application libraries contained inside the image.

The configured scan checks:

```
scan-type: image
vuln-type: os,library
severity: HIGH,CRITICAL
ignore-unfixed: true
exit-code: '1'
```

The initial implementation pulled the upstream crAPI workshop image:

```
crapi/crapi-workshop:latest
```

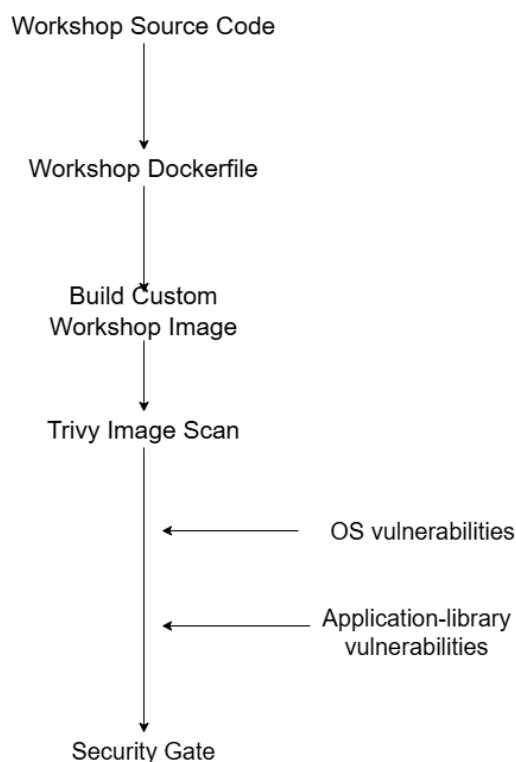
and scanned that image.

However, this approach does not guarantee that the image contains the group's source-code modifications. Therefore, the final implementation must build the workshop image from:

```
crAPI/services/workshop/Dockerfile
```

and scan that locally built image.

The intended final flow is:



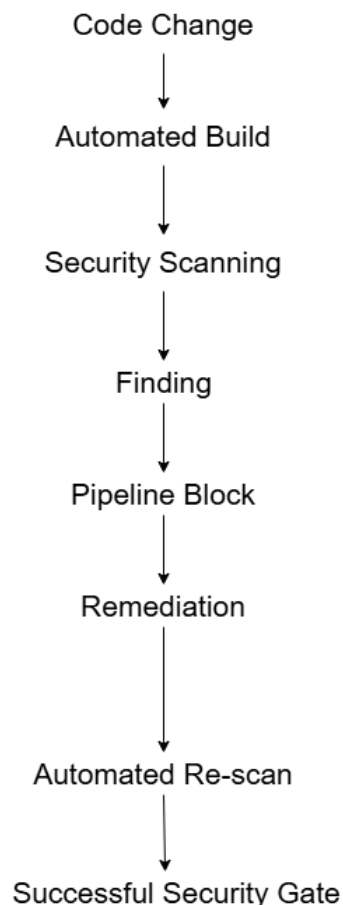
6.7. CI/CD Security Summary

The implemented pipeline integrates security checks into the software delivery process rather than treating security testing as a separate manual activity. Semgrep provides static source-code analysis, Trivy performs dependency and container vulnerability scanning, and Gitleaks checks for accidentally committed secrets.

The initial pipeline execution successfully demonstrated the operation of the security gates. Semgrep identified nine blocking source-code findings, while the Trivy filesystem scan identified 127 dependency vulnerabilities. Gitleaks completed successfully without detecting secrets.

The final pipeline implementation extends this approach by building the affected workshop service from the repository source and scanning the resulting Docker image. This ensures that the container security gate evaluates the same application version that has been modified and tested by the project team.

The final evidence will therefore demonstrate the complete DevSecOps feedback loop:



7. Secrets Management

The repository includes an `.env.example` containing placeholder values for database users/passwords and application secrets. Its `.gitignore` excludes `.env` and common private-key formats. This is a sound starting convention, but the available files do not demonstrate how GitHub Actions secrets are provisioned to a deployment or how the running application receives production-like secrets. The final report should state that example values are not credentials, confirm that a real `.env` file is untracked, and describe the actual secret-injection mechanism used. At minimum, CI credentials must use encrypted GitHub Actions secrets or short-lived built-in tokens, and runtime secrets should be supplied outside source control. Do not include actual values in the report, screenshots, logs, or repository. Review current tracked files and presented history for accidental credentials; rotate any real credential that was committed.

7.1. Local Development Secrets

Development secrets are stored outside the tracked source code using a local `.env` file. The `.env` file is excluded from Git tracking using `.gitignore`, while `.env.example` contains only placeholder values required to explain the configuration structure.

No real passwords, API keys or private credentials are included in the repository or report.

7.2. GitHub Actions Secrets

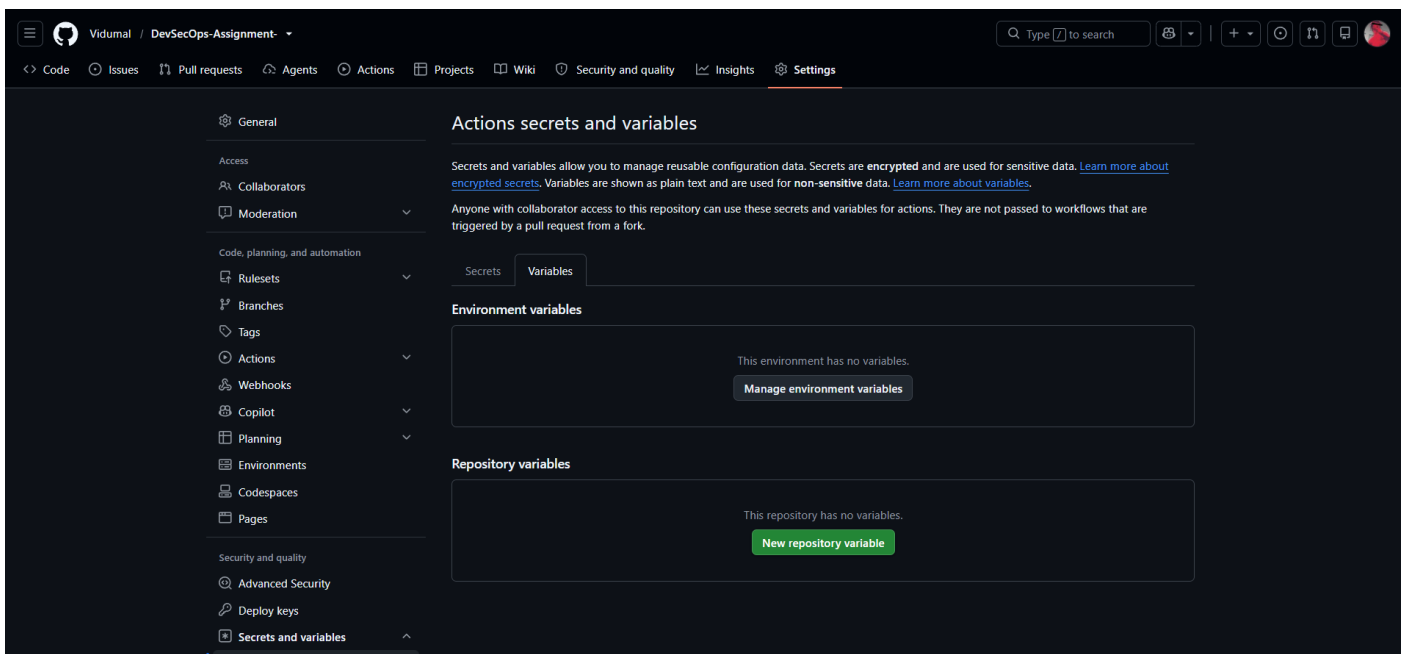
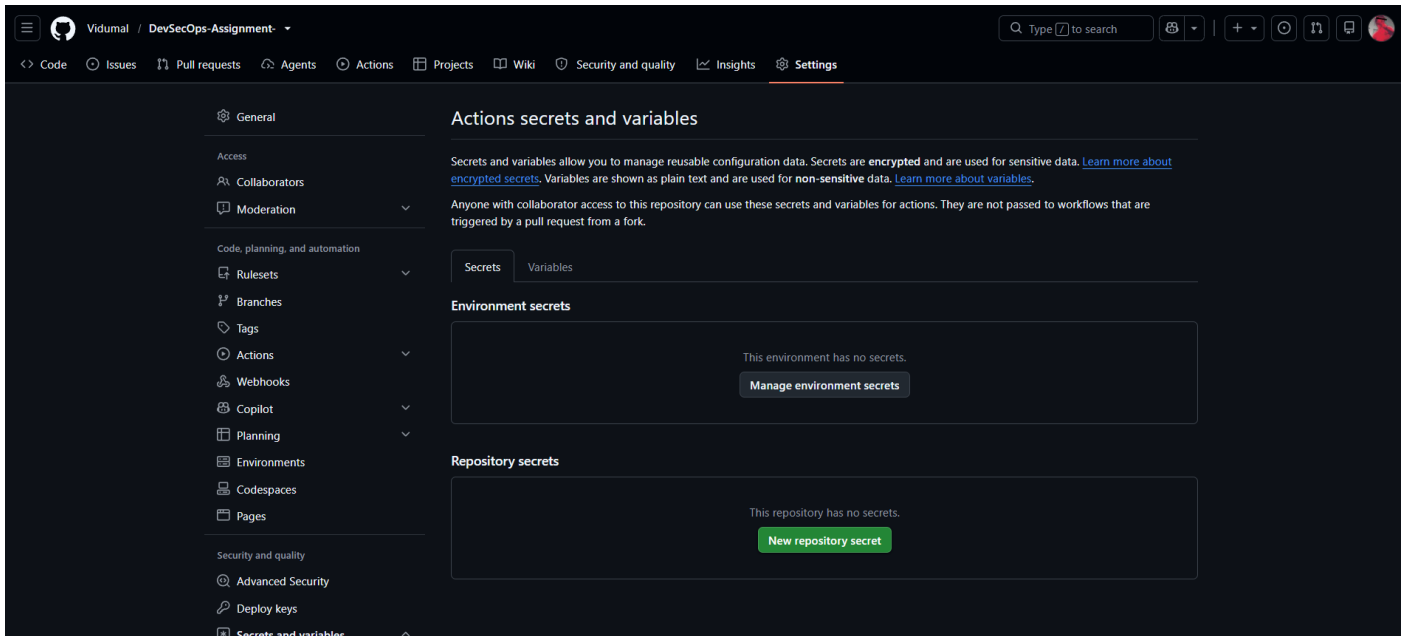
Secrets required by CI/CD are configured using GitHub Actions encrypted repository secrets. Secret names are referenced by the workflow without exposing their values in source code.

Examples include database passwords, application secrets and service credentials. The actual values are not stored in the repository and are not displayed in screenshots or CI logs.

7.3. Secret Injection

During CI execution, secrets are supplied to the workflow through GitHub's encrypted secret mechanism. The workflow references the secret using the GitHub Actions expression syntax and passes it only to the step requiring it.

This separates secret material from application source code and reduces the risk of accidental disclosure through version control.



8. Industry Trend and Case Study

8.1. SolarWinds Supply-Chain Compromise

The SolarWinds Orion supply-chain compromise illustrates why security must cover the build and distribution path as well as application code. CISA’s Alert AA20-352A described compromises affecting government, critical infrastructure, and private-sector organizations and identified the Orion software supply chain as one initial access vector [1]. The relevant lesson for this project is that a security scan of source code alone cannot prove that the deployed artifact corresponds to reviewed source or that its dependencies and build path are trustworthy. The project maps this lesson to source, dependency, secret, and image checks. Those gates only protect delivery when they examine the artifact built from the reviewed commit. Pin inputs, protect branches, tag builds reproducibly, and demonstrate a failing threshold. The current Compose-pull workflow leaves a gap: an upstream image scan does not validate the group’s modified source [2].

8.2. Lessons Applied to This Project

SolarWinds lesson	Project control
Source integrity	Git repository + review
Dependency risk	Trivy filesystem scan
Secret exposure	Gitleaks
Artifact integrity	Build image from repository source
Container vulnerabilities	Trivy image scan
Automated enforcement	GitHub Actions security gates

9. Reflection and Future Improvements

The project showed that selecting vulnerabilities is only the start. Each remediation needs a reproducible baseline, a reviewable code change, and a retest under the same conditions. Authorization tests need identities for both unauthorized and legitimate access. SSRF testing also highlights why the Docker network boundary must be explicit: the backend can reach a service that an external caller cannot.

With more time, the group would add automated regression tests for each security correction, generate a software bill of materials, pin container base images by digest, produce build provenance, and restrict egress from services that do not need outbound access. It would add least-privilege database accounts, centralize structured security logs, and use a dedicated secret manager for runtime deployment. The current workflow should also separate source and image scanning clearly and publish a compact security summary as a CI artifact. These are proposed improvements; they should be presented as future work unless implemented and evidenced.

10. Contributions and AI Use

student ID	Contribution to complete and verify	Signature
IT24102736	SQL injection, verify baseline and fixed coupon endpoint; capture the same payload/response on both versions; locate query fix; attach before/after SAST evidence.	
IT24102746	Unauthenticated service-request access, capture no-token baseline and fixed retest; record status/data exposure; locate authentication fix; attach relevant scan evidence.	
IT24103630	SSRF, capture MailHog baseline request/response and fixed retest; locate URL validation/allowlist or destination-control fix; attach relevant scan evidence	
IT24103854	BOLA/IDOR, capture cross-owner baseline and fixed retest plus authorized-owner control case; locate ownership check; attach relevant scan evidence.	

11. Conclusion

This project demonstrated the integration of security practices into a DevSecOps software delivery lifecycle using OWASP crAPI. Four application vulnerabilities were analysed using STRIDE-based threat modelling and risk assessment, followed by controlled exploitation, source-code remediation and regression testing.

The final implementation integrates Semgrep, Trivy and Gitleaks into a GitHub Actions pipeline. The affected workshop service is built from the project source before container security scanning, ensuring that the scanned artifact corresponds to the reviewed application version.

The project also demonstrates secrets management through separation of credentials from source control and the use of protected CI/CD secret mechanisms. The industry case study demonstrates why security controls must cover source code, dependencies, secrets and build artifacts rather than relying on application testing alone.

Overall, the implementation demonstrates a practical DevSecOps feedback loop in which vulnerabilities are identified, security controls block unacceptable conditions, remediation is applied, and the resulting build is re-tested before being considered secure.

12. References

- [1] Cybersecurity and Infrastructure Security Agency (CISA), "AA20-352A: Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations," Dec. 2020. [Online]. Available: <https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-352a>. Accessed: Sep. 28, 2026.
- [2] Enduring Security Framework, "Securing the Software Supply Chain: Recommended Practices for Developers," CISA/NSA, 2023. [Online]. Available: https://www.cisa.gov/sites/default/files/2023-12/ESF_SECURING_THE_SOFTWARE_SUPPLY_CHAIN_DEVELOPERS.pdf. Accessed: Sep. 28, 2026.
- [3] OWASP Foundation, "OWASP crAPI." [Online]. Available: <https://owasp.org/projects/crapi>. Accessed: Sep. 28, 2026.
- [4] OWASP, "crAPI: Completely Ridiculous API," source repository and project documentation. [Online]. Available: <https://github.com/OWASP/crAPI>. Accessed: Sep. 28, 2026.
- [5] IE3142 DevOps Security, "Building and Securing a DevSecOps Pipeline," Continuous Assessment assignment brief, 2026.